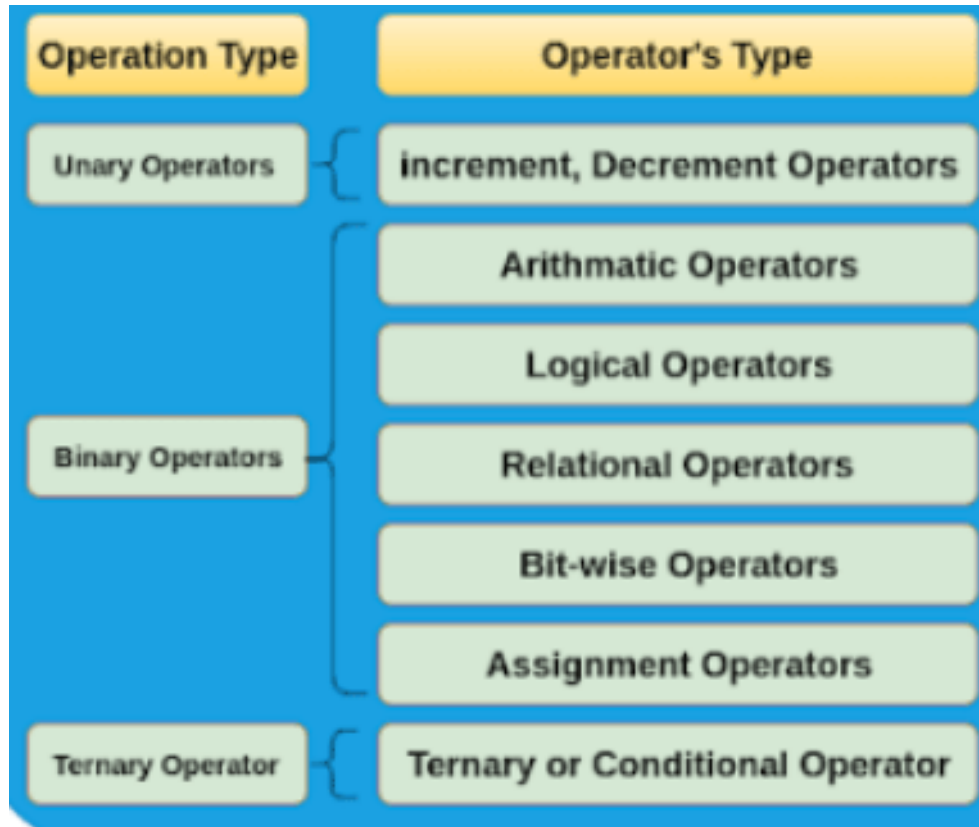


# **BCSE102L Structured and Object Oriented Programming**

# Operators in C

C supports a rich set of built-in operators.

An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.



# Operators in C

## Arithmetic Operators

Operates on any built-in data types

Example operators:

- 1) **+** (**Plus**) – Give sum as a result. Example:  $3+2=5$
- 2) **-** (**Minus**) – Give the difference as a result. Example:  $4-2=2$
- 3) **\*** (**Asterisk**) – Used for multiplication and give the product as a result. Example:  $4*2=8$
- 4) **/** (**Forward slash**) – Used for division and give quotient as a result. Example:  $4/2=2$
- 5) **%** (**modulus**) – Give the remainder as a result. Example:  $7\%2=1$

# Operators in C

## Relational Operators

To compare two quantities, and depending on their relation, take certain decisions.

Eg.  $a > b$

| Operator | Use                      |
|----------|--------------------------|
| <        | Less than                |
| <=       | Less than or equal to    |
| >        | Greater than             |
| >=       | Greater than or equal to |
| ==       | Equal                    |
| !=       | Not equal                |

# Operators in C

## Logical Operators

The logical operators are used to test more than one conditions and make decisions.

### Example Operators:

Logical AND            &&

Logical OR            ||

Logical NOT           !

Eg. `a>b && a>c`

# Operators in C

## Assignment Operators

It is used to assign the results of an expression to a variable.

**Syntax:**

**var op = exp;**

**var = var op (exp);**

**Example:**

**x += y+1**

**x = x + (y+1);**

var – variable, op – binary arithmetic operator, exp - expression

# Operators in C

## Increment and Decrement Operators

1. Increment operators (++): Example: (++x)
2. Decrement operators (--): Example: (--m)

These operators can be prefix operator or postfix operator.

**Prefix increment operator:** ++m is equivalent to m=m+1 or m+=1

**Postfix increment operator:** a = m++ -j+10, in this case the expression is evaluated first using the original value of the variable and then variable is incremented.

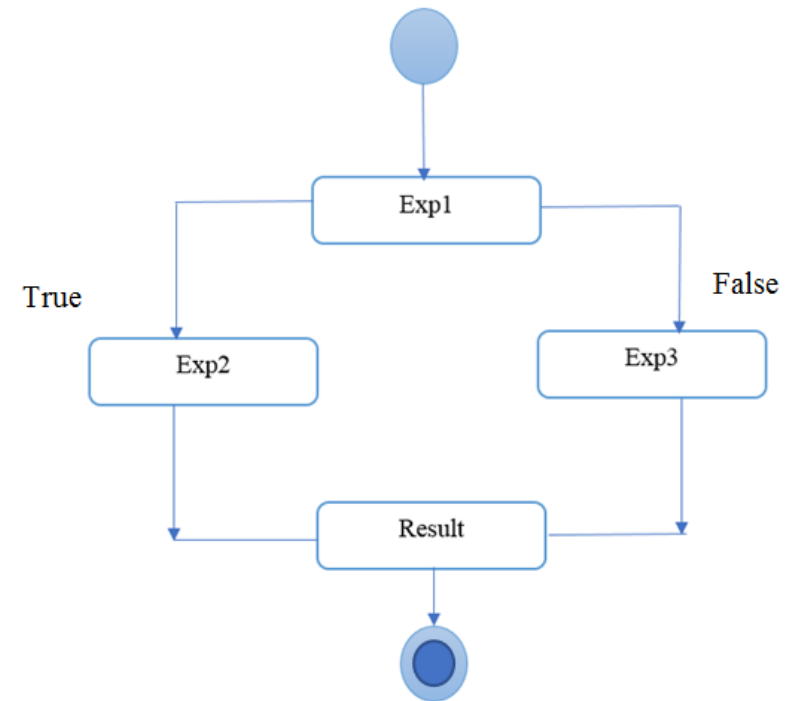
# Operators in C

## Conditional Operator

A ternary operator pair “:?” is available in C to construct conditional expressions of the form:

**Syntax:** `exp1 ? exp2 : exp3`

**Example:** `x = (a > b) ? a : b;`





# Operators in C

## Bitwise Operator

Used for manipulation of data's at bit level.

Used for testing the bits, or shifting them right or left

| operator's | meaning of operators       |
|------------|----------------------------|
| &          | <b>Bitwise AND</b>         |
|            | <b>Bitwise OR</b>          |
| ^          | <b>Bitwise XOR</b>         |
| ~          | <b>Bitwise Complement</b>  |
| <<         | <b>Bitwise left shift</b>  |
| >>         | <b>Bitwise right shift</b> |

# Operators in C

## Special Operators

1. The comma operator – used to link the related expressions together  
Example: `value = (x=10, y=5, x+5);`
2. The sizeof operator – returns the number of bytes the operand occupies  
Example: `m = sizeof(sum);`
3. Pointer operator (& and \*)
4. Member selection operator (. and ->)

# Operator Precedence and Associativity

C has precedence associated with it.

This precedence is used to determine how an expression involving more than one operator is evaluated

The operators at the highest level are evaluated first

The operators at the same precedence are evaluated either from left to right or from right to left, depending on the level. This is known as associativity property of an operator.

Consider the conditional statement,

**if (x == 10 + 15 && y < 10)**

**if (x == 25 && y < 10)**

// Precedence rule: addition operator has highest priority than logical and relational operators

# Precedence of Arithmetic Operators

| Category       | Operator                      | Associativity |
|----------------|-------------------------------|---------------|
| Postfix        | () [] -> . ++ --              | Left to right |
| Unary          | + - ! ~ ++ -- (type)* &sizeof | Right to left |
| Multiplicative | * / %                         | Left to right |
| Additive       | + -                           | Left to right |
| Shift          | << >>                         | Left to right |
| Relational     | <<= >>=                       | Left to right |
| Equality       | == !=                         | Left to right |
| Bitwise AND    | &                             | Left to right |
| Bitwise XOR    | ^                             | Left to right |
| Bitwise OR     |                               | Left to right |
| Logical AND    | &&                            | Left to right |

|             |                                   |               |
|-------------|-----------------------------------|---------------|
| Logical OR  |                                   | Left to right |
| Conditional | ?:                                | Right to left |
| Assignment  | = += -= *= /= %= >>= <<= &= ^=  = | Right to left |
| Comma       | ,                                 | Left to right |

# Precedence of Arithmetic Operators

**An example:**

```
#include <stdio.h>
int main()
{
    float a,b,c,x;
    a=9; b=12; c=3;
    x= a - b / 3 + c * 2 - 1;
    printf("x=%f", x);
    return 0;
}
```

**OUTPUT?**

# How to read data from keyboard?

**Scanf("control string", &variable1, &variable2,..);**

**Scanf("%d", &number);**

**Example:**

```
#include <stdio.h>
int main()
{
    int a,b,sum;
    printf("Enter the value for A and B: ");
    scanf("%d %d", &a, &b);
    sum=a+b;
    printf("Sum=%d", sum);
    return 0;
}
```

# Simple C Programs

## Program: To print a text

```
#include <stdio.h>
int main()
{
printf("Hello, World! \n");
return 0;
}
```

## Program: Display Variable Value

```
#include <stdio.h>

int main()
{
    int a=50;
    int b=5;
    //a=50, b=5;
    int c=a+b;
    printf("Sum of %d and %d is: %d", a, b, c);
}
```

```
#include <stdio.h>
main()
{
    int number;
    float amount;
    number=100;
    amount=32.75+53.50;
    printf("%d", number);
    printf("\n%4.2f", amount);
    //return 0;
}
```



## Program: Find the area of the square

```
#include<stdio.h>

int main() {
    int side, area;
    printf("\n Enter the Length of Side : ");
    scanf("%d", &side);
    area = side * side;
    printf("\n Area of Square : %d", area);
    return (0);
}
```

# Program to calculate simple interest.

```
#include<stdio.h>
#include<conio.h>
int main()
{
float p,rate,time,si;
printf("Enter principal amount : ");
scanf("%f", &p);
printf("Enter rate of interest : ");
scanf("%f", &rate);
printf("Enter time period in year : ");
scanf("%f", &time);
/*calculating simple interest*/
si=(p*time*rate)/100;
printf("\nSimple Interest = %f",si);
getch();
return 0;}
```

# Program to swap two numbers.

```
#include <stdio.h>
int main()
{
    int A, B, temp;
    printf("Please enter the 1st number : ");
    scanf("%d",&A);
    printf("\nPlease enter the 2nd number : ");
    scanf("%d",&B);
    printf("\nBefore swapping:\n");
    printf("A - %d \nB - %d", A, B);

    temp = A;
    A = B;
    B = temp;

    printf("\nAfter swapping:\n");
    printf("A - %d \nB - %d", A, B);
    return 0;
}
```